



第五章 数组和广义表

杨震 计算机学院

yangzhen@bupt.edu.cn

本章内容

BUPT

- 5.1 数组和线性表的关系以及数组的运算
- 5.2 数组的顺序存储结构
- 5.3 矩阵的压缩存储
- 5.4 广义表的定义和表示方法
- 5.5 广义表的存储结构
- 5.6 广义表的递归算法

5.1 数组和线性表的关系以及数组的运算

BUPT

任何数组A都可以看作一个线性表

$$A=(a_1, a_2, \dots, a_i, \dots a_n)$$

二维数组 $n \times m$ 时,

a_i 是数组中第 i 行所有元素, $1 \leq i \leq n$, 表中每一个元素是一个一维数组;

三维数组时,

表中每一个元素是一个二维数组;

n 维数组时,

表中每一个元素是一个 $(n-1)$ 维数组

$$A_{m \times n} = \begin{bmatrix} \overbrace{(a_{11} \quad a_{12} \quad \dots \quad \dots \quad a_{1n})} & \\ \overbrace{(a_{21} \quad a_{22} \quad \dots \quad \dots \quad a_{2n})} & \\ \overbrace{(\dots \quad \dots \quad \dots \quad \dots \quad \dots)} & \\ \overbrace{(a_{m1} \quad a_{m2} \quad \dots \quad \dots \quad a_{mn})} & \end{bmatrix}$$

线性表的扩展，其数据元素本身也是线性表

数组的特点

- 数组中各元素都具有统一的类型
- 可以认为， d 维数组的非边界元素具有 d 个直接前趋和 d 个直接后继
- 数组维数确定后，数据元素个数和元素之间的关系不再发生改变，适合于顺序存储
- 每组有定义的下标都存在一个与其相对应的值

在数组上的基本操作

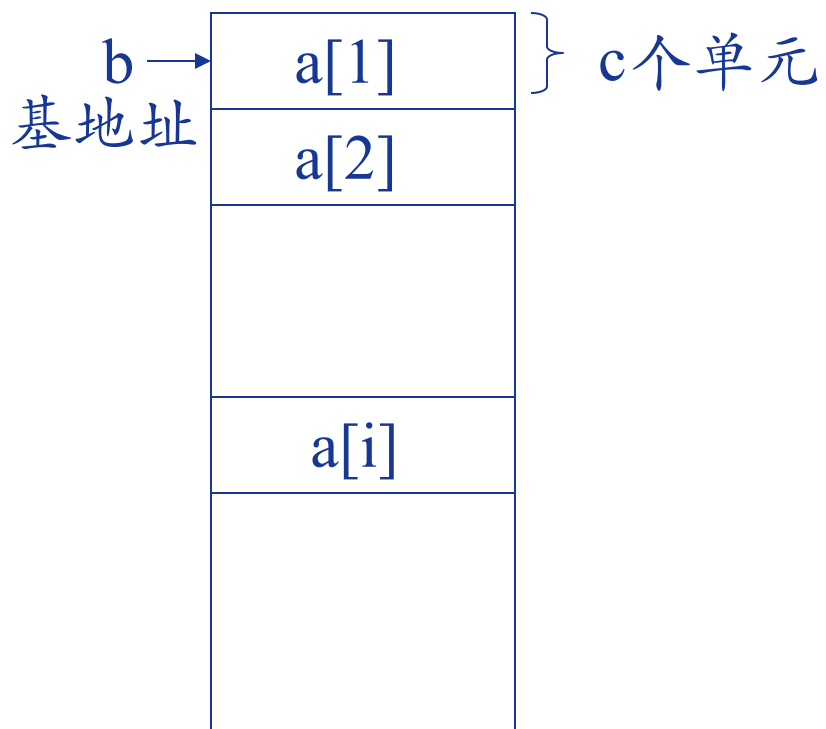
- 给定一组下标，取得相应的数据元素值
- 给定一组下标，修改相应的数据元素值

5.2 数组的顺序存储结构

BUPT

一维数组

elemtype a[1,...,n]



$$\text{loc}(a[i]) = b + (i-1)c$$

$$= b - c + i * c$$

$$= C_0 + C_1 * i$$

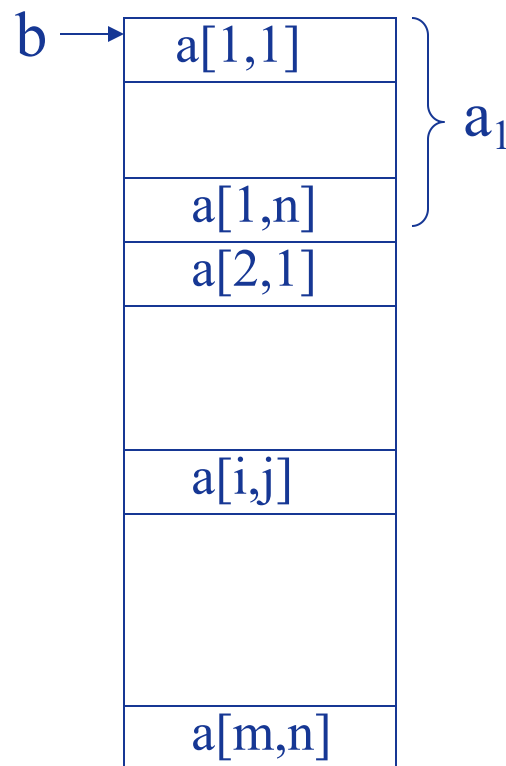
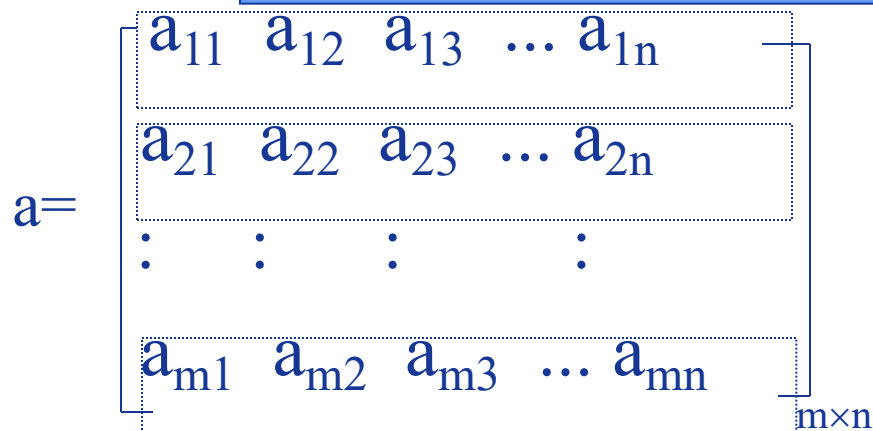
$$\begin{cases} C_1 = c \end{cases}$$

$$\begin{cases} C_0 = b - C_1 \end{cases}$$

二维数组

BUPT

```
elemtype a[1..m][1..n];
```



$$\begin{aligned}\text{loc}(a[i,j]) &= b + [(i-1)*n + (j-1)]*c \\ &= C_0 + C_1*i + C_2*j\end{aligned}$$

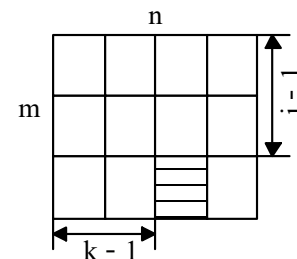
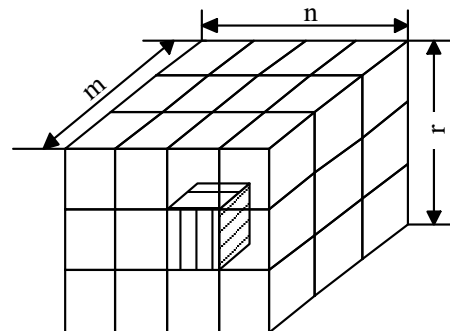
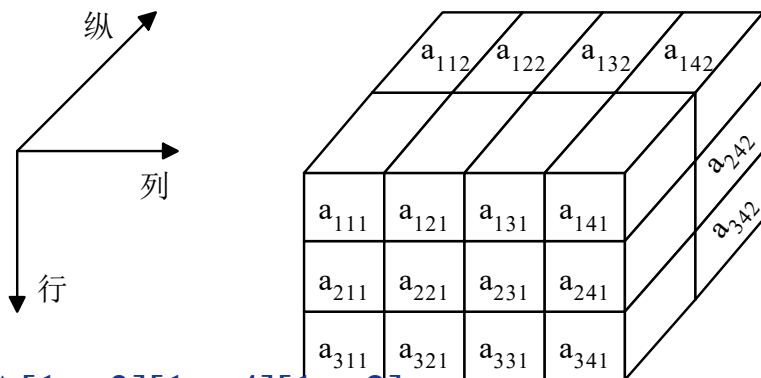
$$\begin{cases} C_2 = c \\ C_1 = n * C_2 \\ C_0 = b - C_1 - C_2 \end{cases}$$

注：Pascal、C语言以行序为主序
Fortran 语言以列序为主序

三维数组

BUPT

三维数组A (1..r, 1..m, 1..n) 可以看成是r个m×n的二维数组



$A[1,...,3][1,...,4][1,...,2]$

行优先时存放顺序: $a_{111}, a_{112}, a_{121}, a_{122}, \dots, a_{331}, a_{332}, a_{341}, a_{342}$

$$\text{Loc } [i, j, k] = \text{Loc } [1, 1, 1] + [(i-1) \times m \times n + (j-1) \times n + (k-1)] \times c$$

思考:如果三维数组为 $A[c_1, d_1][c_2, d_2][c_3, d_3]$,每个数据元素占 c 个单元
则 $\text{Loc } [j_1, j_2, j_3] = ?$

$$\begin{aligned} \text{Loc } [j_1, j_2, j_3] &= \text{Loc } [c_1, c_2, c_3] + c \times (d_2 - c_2 + 1) \times (d_3 - c_3 + 1) \times (j_1 - c_1) \\ &+ c \times (d_3 - c_3 + 1) \times (j_2 - c_2) + c \times (j_3 - c_3) \end{aligned}$$

```
elemtype a[1..n1][1..n2] ,..., [1..nd];
```

$$\text{loc}(a[i_1, i_2, \dots, i_d]) = C_0 + C_1 * i_1 + C_2 * i_2 + \dots + C_d * i_d$$

$$\begin{cases} C_d = c; \\ C_{t-1} = C_t * n_t \quad (1 < t \leq d) \\ C_0 = b - C_1 - C_2 - \dots - C_d \\ b = \text{loc}(a[1, 1, \dots, 1]) \end{cases}$$

数组是一种随机存取结构。

思考：

```
elemtype a [3..7][ 0..9][1..6][5..10];
```

设每数组元素占用8个存储单元，求按低下标优先（类似行优先）顺序存储时， $\text{loc}(a[6, 3, 3, 7])=?$

C语言中的多维数组表示

BUPT

①定义和引用多维数组的唯一方法就是使用数组的数组。即多维数组看成是某种对象的一维数组，数组的元素可以是另一个数组。

②C语言定义多维数组时，最右边的下标先变化，即行优先顺序。

③C语言中任何一维数组均可作为函数的实参，但无法向函数传递普通的多维数组。这是因为我们需要知道每一维的长度，为地址运算提供正确的单位长度。因此必须提供除最左边一维外的所有维的长度，把实参限制在除最左边一维外的所有维都必须与形参匹配的数组。

【例】char carrot[10][20]; 或
typedef char vegetable[20];
vegetable carrot[10];

上述两种形式，访问carrot[i][j]时，编译器编译时都解释成：
((carrot+i)+j)

【例】定义函数fun(int a[][3][5])下面调用方式可行的有哪些？

1. int b[10][3][5]; fun(b);
2. int c[10][5][5]; fun(b);
3. int c[999][3][5]; fun(c);
4. int d[999][3][6]; fun(d);

设矩阵 $a[1,..3][1,..3]$ 如下，执行下列语句后，矩阵 $c[1,..3][1,..3]$ 和 $a[1,..3][1,..3]$ 的结果分别是什么？

1) for (i=1; i<=3;i++)
 for(j=1; j<= 3;j++)
 c[i,j] = a[a[i,j],a[j,i]]

(2) for (i=1; i<=3;i++)
 for(j=1; j<= 3;j++)
 a[i,j] = a[a[i,j],a[j,i]]

$$c = \begin{bmatrix} 3 & 3 & 1 \\ 1 & 1 & 1 \\ 3 & 3 & 2 \end{bmatrix}$$

$$a = \begin{bmatrix} 3 & 3 & 1 \\ 1 & 1 & 3 \\ 3 & 3 & 3 \end{bmatrix}$$

【示例】 设计一个算法，将含有 n 个元素的整形数组 $A[0..n-1]$ 的元素循环右移 m 位($1 \leq m \leq n$)，要求空间复杂度为 $O(1)$ 。

例如假设 $A[0..5]$ 是1 2 3 4 5 6；循环右移2位，则数组变为5 6 1 2 3 4。

思路：假设将数组 A 表示为 XY ， X 表示前 $n-m$ 个元素， Y 表示后 m 个元素。则循环右移 m 位后得到 YX 。那如何得到 YX ？

可①置逆 X ，得到 $X^{-1}Y$ ；②置逆 Y ，得到 $X^{-1}Y^{-1}$ ；③整体置逆 $(X^{-1}Y^{-1})^{-1}=YX$ 。

```
Void Reverse(int A[], int i, int j) //置逆A[i..j]
{
    int k, tmp;
    for (k=0; k<(j-i+1)/2; k++)
    {
        tmp = A[i+k];
        A[i+k] = A[j-k];
        A[j-k] = tmp;
    }
}
```

```
void RightMove(int A[], int n, int m)
{
    Reverse(A, 0, n-m-1);
    Reverse(A, n-m, n-1);
    Reverse(A, 0, n-1);
}
```

5.3 矩阵的压缩存储

BUPT

目的是节省空间。

5.3.1 对称矩阵

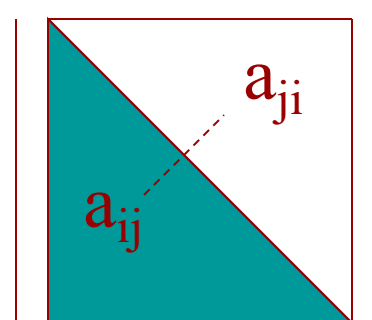
[特点] 在 $n \times n$ 的矩阵 a 中, 满足如下性质:

$$a_{ij} = a_{ji} \quad (1 \leq i, j \leq n)$$

[存储方法] 只存储下(或者上)三角(包括主对角线)的数据元素。共占用 $n(n+1)/2$ 个元素空间。

sa	a_{11}	a_{21}	a_{22}	a_{31}		$a_{ij}(a_{ji})$		a_{nn}
k	1	2	3	4				$n(n+1)/2$

$$k = \begin{cases} i(i-1)/2 + j & \text{当 } i \geq j \\ j(j-1)/2 + i & \text{当 } i < j \end{cases}$$



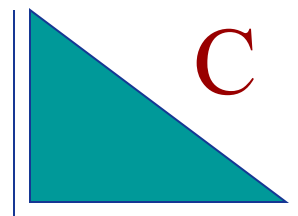
5.3.2 三角矩阵

BUPT

[特点] 对角线以下(或者以上)的数据元素(不包括对角线)全部为常数c。



上三角矩阵



下三角矩阵

[存储方法] 重复元素c共享一个元素存储空间，共占用 $n(n+1)/2+1$ 个元素空间: $sa[1.. n(n+1)/2+1]$

上三角矩阵

下三角矩阵

$$k = \begin{cases} (i-1) \times (2n-i+2)/2 + j - i + 1 & i \leq j \\ n(n+1)/2 + 1 & i > j \end{cases}$$

$$k = \begin{cases} i \times (i-1)/2 + j & i \geq j \\ n(n+1)/2 + 1 & i < j \end{cases}$$

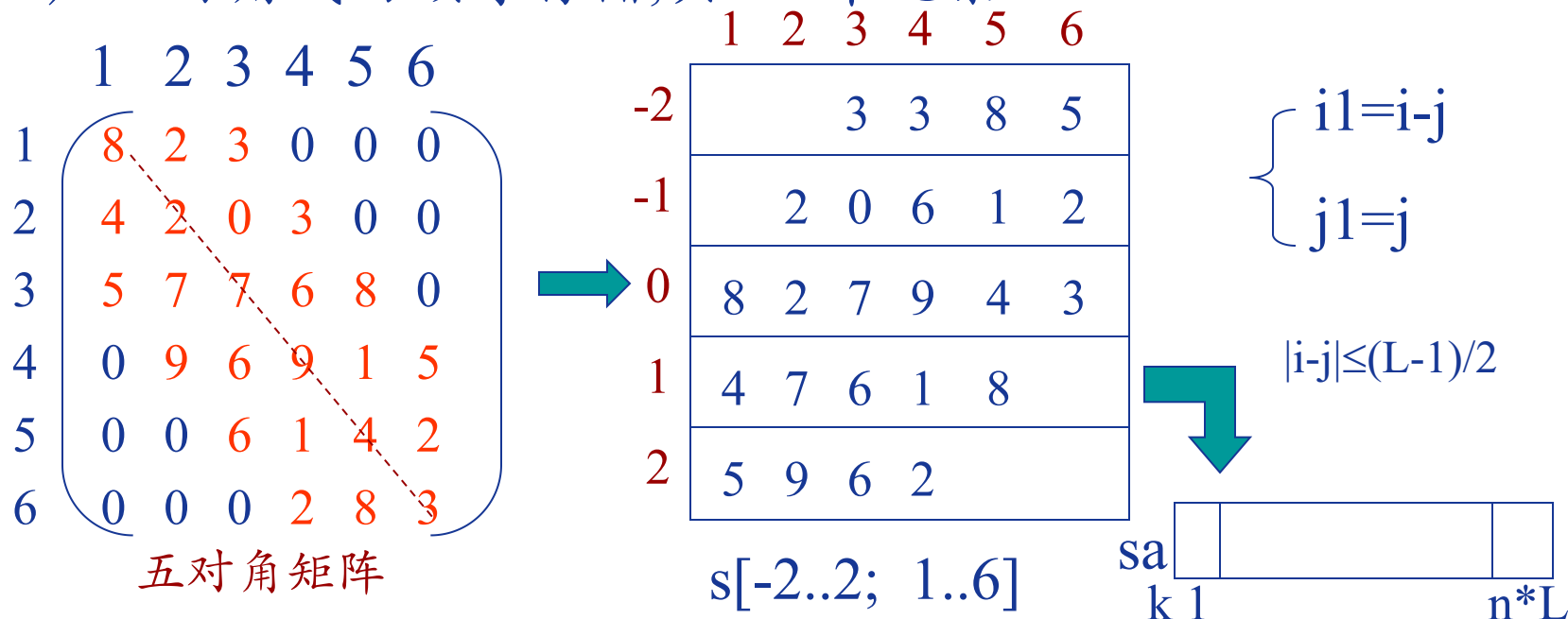
5.3.3 带状矩阵（对角矩阵）

BUPT

[特点] 在 $n \times n$ 的方阵中，非零元素集中在主对角线及其两侧共 L (奇数)条对角线的带状区域内—— L 对角矩阵。

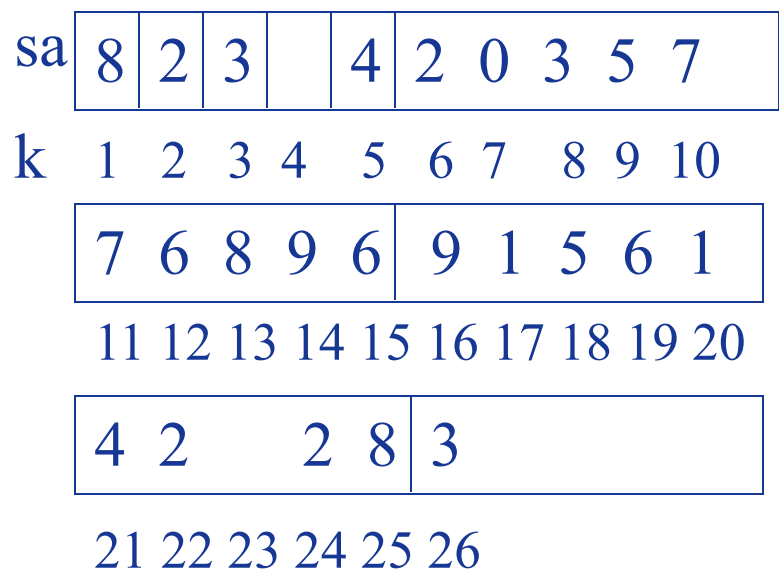
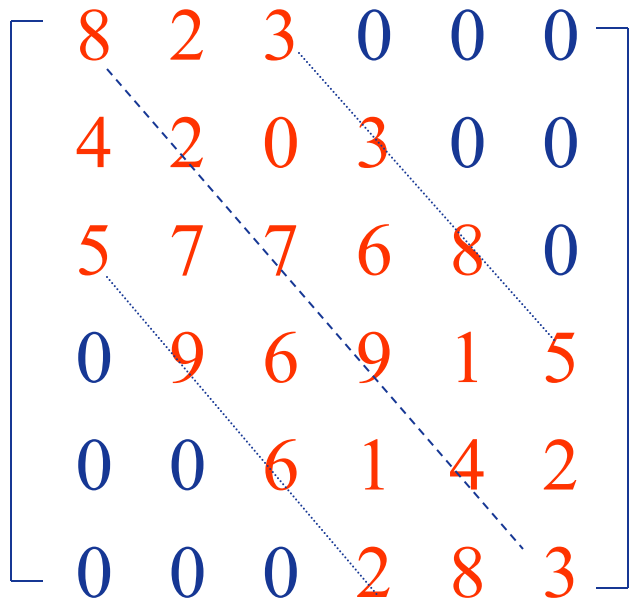
[存储方法] 只存储带状区内的元素。

1) 以对角线的顺序存储,共 $n * L$ 个元素。



2) 只存储带状区内的元素。除首行和末行，按每行 L 个元素，共 $(n-2)L+(L+1)$ 个元素。
 $sa[1..(n-1)L+1]$

存储地址计算: $k=(i-1)L+1+(j-i)$
 $|i-j|\leq(L-1)/2$



5.3.4 稀疏矩阵

[特点] 大多数元素为零。

[常用存储方法] 记录每一非零元素 (i,j,a_{ij})

——节省空间，但丧失随机存取功能

❖ 顺序存储：三元组表

❖ 链式存储：十字(正交)链表

15	0	0	22	0	-15
0	11	3	0	0	0
0	0	0	-6	0	0
0	0	0	0	0	0
91	0	0	0	0	0
0	0	28	0	0	0

5.3.4.1 三元组表

BUPT

[类型定义]

```
#define u={矩阵中非零元素最大个数};

typedef struct tuple
{
    int i,j;
    elemtype v;
} TupleNode;

struct sparmattp
{
    int m,n,t; \\m:行数; n:列数; t:非零元素数目
    TupleNode data[1..u];
}

typedef struct sparmattp TSMatrix;
```

	i	j	v
1	1	1	15
2	1	4	22
3	1	6	-15
4	2	2	22
5	2	3	3
6	3	4	-6
7	5	1	91
8	6	3	28

[算法例] 求转置矩阵

BUPT

```
算法1 {  $O(n*t)$  }  
void trans_sparmat(TSMatrix a, TSMatrix b)  
{  
    b.m=a.n; b.n=a.m; b.t=a.t;  
    if (b.t > 0)  
    {  
        q=1;      {指示b中存放数据的位置, 初值为1}  
        for (col=1; col<=a.n; col++) {对a扫描a.n趟}  
            for (p=1; p<=a.t; p++) {对a.t个数据检查}  
                if (a.data[p].j==col) {满足当前扫描列号}  
                {  
                    b.data[q].i=a.data[p].j; {向b赋值}  
                    b.data[q].j=a.data[p].i;  
                    b.data[q].v=a.data[p].v;  
                    q=q+1; {修正q值}  
                }  
            }  
        }  
    }  
}
```

算法2 快速转置法 { $O(n+t)$ }

BUPT

引入两个辅助向量:

- $\text{num}[1,..,a.n]$: a 中每列的非零元素个数
- $\text{cpot}[1,..,a.n]$: a 中每列的第一个非零元素在 b 中的位置

a 中的列号

col num[col] cpot[col]

1	2	1
2	1	3
3	2	4
4	2	6
5	0	7
6	1	8

a	i	j	v		b	i	j	v
1	1	1	15	→	1	1	1	15
2	1	4	22	→	2	1	5	91
3	1	6	-15	→	3	2	2	22
4	2	2	22	→	4	3	2	3
5	2	3	3	→	5	3	6	28
6	3	4	-6	→	6	4	1	22
7	5	1	91	→	7	4	3	-6
8	6	3	28	→	8	6	1	-15

转置

$\text{cpot}[1]=1$

$\text{cpot}[\text{col}] = \text{cpot}[\text{col}-1] + \text{num}[\text{col}-1]$
($2 \leq \text{col} \leq n$)

```

void fast_transpos(TSMatrix a, TSMatrix b)
{
    b.m=a.n; b.n=a.m; b.t=a.t;
    if (b.t > 0)
    {
        for(col=1; col<=a.n; col++)
            num[col]=0;      {num清零}
        for (t=1; t<=a.t; t++)      {对a.t个非零元素}
            num[a.data[t].j]=num[a.data[t].j]+1; {按列号计数}
        cpot[1]=1;      {生成cpot}
        for (col=2; col<=a.n; col++)
            cpot[col]=cpot[col-1]+num[col-1];
        for (p=1; p<= a.t; p++)      {对a.t个非零元素转置}
        {
            col=a.data[p].j; q=cpot[col];
            b.data[q].i=a.data[p].j;
            b.data[q].j=a.data[p].i;
            b.data[q].v=a.data[p].v;
            cpot[col]=cpot[col]+1;
        }
    }
}

```

5.3.4.2 十字（正交）链表

BUPT

[特点] 在行、列两个方向上，将非零元素链接在一起。
克服三元组表在矩阵的非零元素位置或个数经常变动时的使用不便。

[结构]

row	col	data
down		right

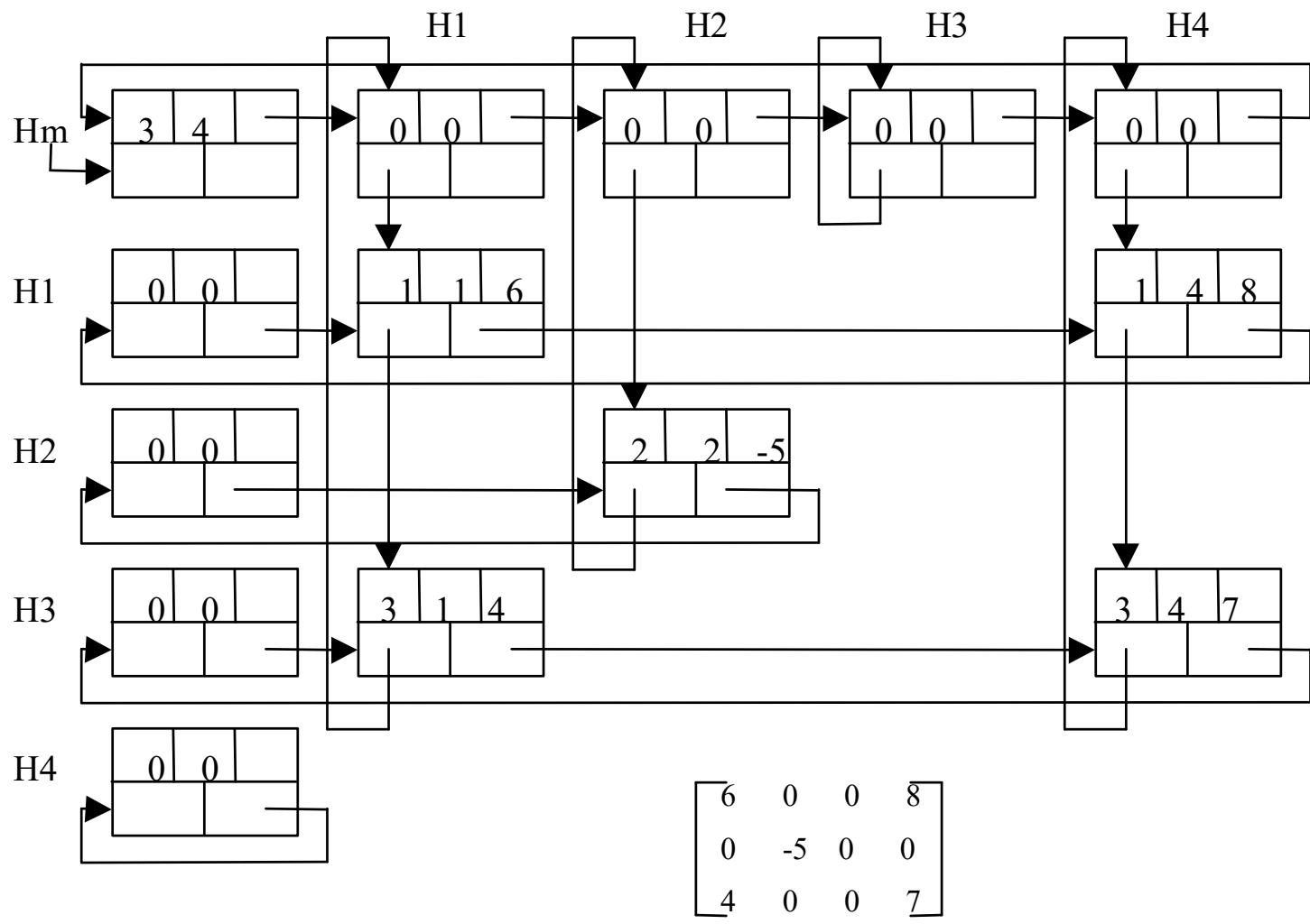
元素结点结构

row	col	next
down		right

头结点结构

十字链表结点结构

```
typedef struct mtxn
{
    int row, col; //行号，列号
    struct mtxn *right, *down;
    union
    {
        int data;
        struct mtxn * next;
    }
}
```



[算法示例] 从终端接收信息建立稀疏矩阵的十字链表

算法思想:

- 1) 读入矩阵的行数 m 和列数 n
- 2) 建立 hm 头结点, 建立 $\max(m,n)$ 个 H 头结点,
并建立上述结点的循环链表
- 3) 读入一个非零元素的行号 r 、列号 c 、值 v
 - 3.1) 建立该元素的结点
 - 3.2) 寻找该结点在行表中的插入位置并插入
 - 3.3) 寻找该结点在列表中的插入位置并插入
 - 3.4) 存在下一个非零元素, 转3; 否则, 结束。

5.4 广义表（列表）的定义和表示方法

概念 广义表是由零个或多个原子或者子表组成的有限序列。可以记作： $LS=(d_1, d_2, \dots, d_n)$

原子：逻辑上不能再分解的元素。

子表：作为广义表中元素的广义表。

逻辑定义 $Lists = (D, R)$

与线性表的关系

- ① 广义表中的元素全部为原子时即为线性表。
- ② 线性表是广义表的特例，广义表是线性表的推广。

广义表的表示方法和相关术语

BUPT

	表长	表深	表头	表尾
$A=()$	0	1	-	-
$B=(a, A)=(a, ())$	2	2	a	$(())$
$C=((a,b), c, d)$	3	2	(a,b)	(c,d)
$D=(a, D)=(a, (a, (a, \dots)))$	2	∞	a	(D)

表的长度 表中的元素（第一层）个数。

表的深度 表中元素的最深嵌套层数。

表头 表中的第一个元素。

表尾 除第一个元素外，剩余元素构成的广义表。

性质 任何一个非空广义表 $LS = (\alpha_1, \alpha_2, \dots, \alpha_n)$ 均可分解为

表头 $\text{Head}(LS) = \alpha_1$ 和

表尾 $\text{Tail}(LS) = (\alpha_2, \dots, \alpha_n)$ 两部分。

示例 $D = (E, F) = ((a, (b, c)), F)$

$\text{Head}(D) = E$ $\text{Tail}(D) = (F)$

$\text{Head}(E) = a$ $\text{Tail}(E) = ((b, c))$

$\text{Head}((b, c)) = (b, c)$ $\text{Tail}((b, c)) = ()$

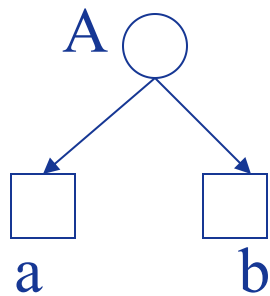
$\text{Head}((b, c)) = b$ $\text{Tail}((b, c)) = (c)$

$\text{Head}((c)) = c$ $\text{Tail}((c)) = ()$

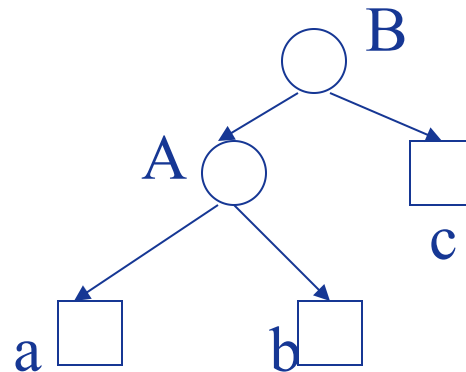


广义表的图形表达方法

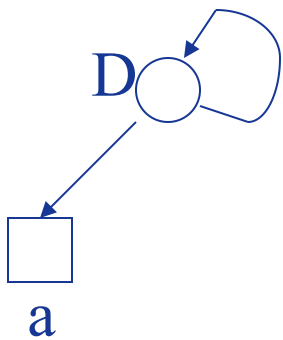
BUPT



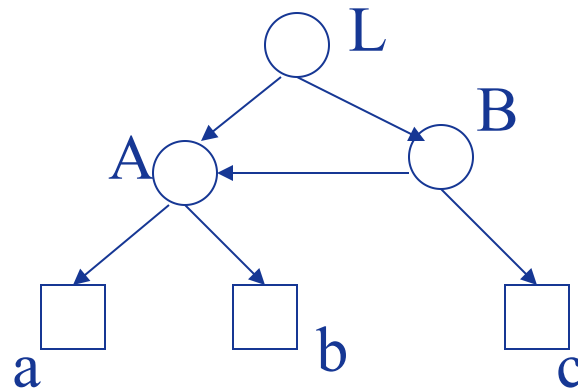
$A=(a,b)$



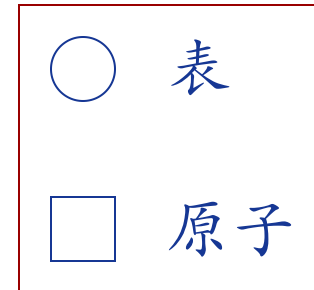
$B=(A,c)$



$D=(a,D)$



$L=(A,B)$



规定所有表都有名字时，广义表的表示方法

例 $A(a,b)$

$B(A(a,b), c)$

$L(A(a,b), B(A(a,b),c))$

$D(a, D(a, D(...)))$

递归表的深度是无穷值，长度是有限值。

广义表结构的分类

① **纯表**：与树型结构对应的广义表。

② **再入表**：允许结点共享的广义表。

③ **递归表**：允许递归的广义表。

递归表 $\supset$ 再入表 $\supset$ 纯表 $\supset$ 线性表

5.5 广义表的存储结构

BUPT

5.5.1 方法1—头尾链表形式

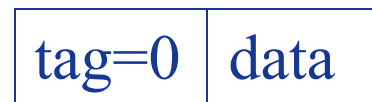
[类型定义]

```
typedef enum {ATOM, LIST} ElemTag;
/*ATOM=0,原子; LIST=1,子表 */
typedef struct GLNode
{
    ElemTag tag;
    /* 标志位tag用来区别原子结点和表结点 */
    union
    {
        elemtype data;
        /* 原子结点的值域data */
        struct { struct GLNode * hp, *tp; } ptr;
        /* 表结点的指针域ptr,
        包括表头和表尾指针域 */
    };
    /* 原子结点的值域atom和
    表结点的指针域ptr的联合体域 */
} *GList;
```

表结点



原子结点



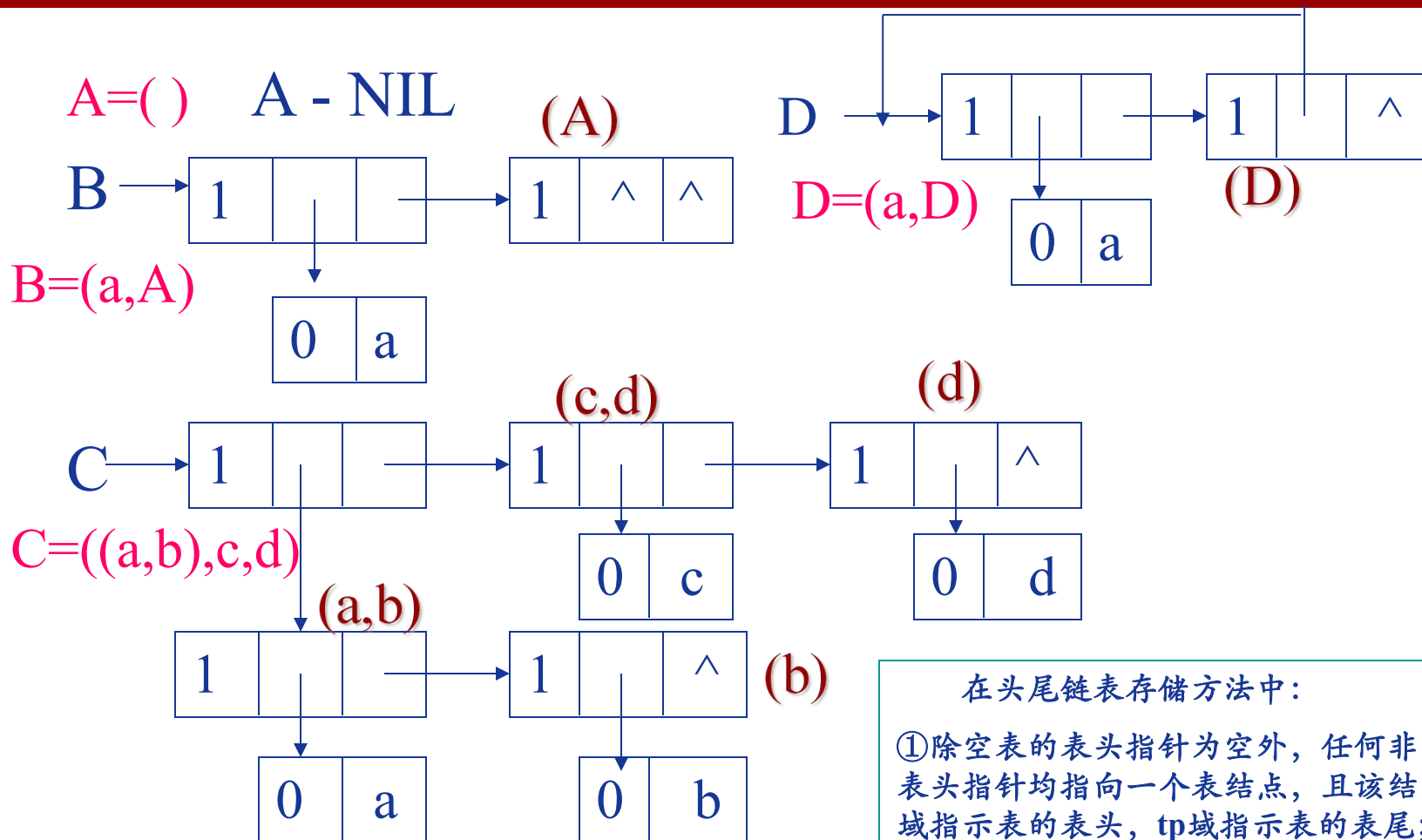
利用
tag来
区分类
型

hp: 指示表头的指针

tp: 指示表尾的指针

[示例]

BUPT



在头尾链表存储方法中:

- ①除空表的表头指针为空外, 任何非空表的表头指针均指向一个表结点, 且该结点的hp域指示表的表头, tp域指示表的表尾;
- ②容易分清列表中的原子和子表所在的层次;
- ③最高层的表结点数即为列表的长度。

5.5.2 方法2—扩展的线性链表形式

[类型定义]

```
typedef enum {ATOM, LIST} ElemTag;
/* ATOM=0, 表示原子; LIST=1, 表示子表
*/
struct GLNode
{
    ElemTag tag;
    union
    {
        AtomType data;
        struct GLNode * hp;
    };
    /* 原子结点的值域atom和
    表结点的表头指针域hp的联合体域 */
    struct GLNode * tp;
};
typedef struct GLNode * GList;
```

表结点

tag=1	hp	tp
-------	----	----

原子结点

tag=0	data	tp
-------	------	----

hp: 指向表头的指针

tp: 指向同一层的下一个结点, 与链表的next指针含义相同

对应着树的孩子兄弟链表

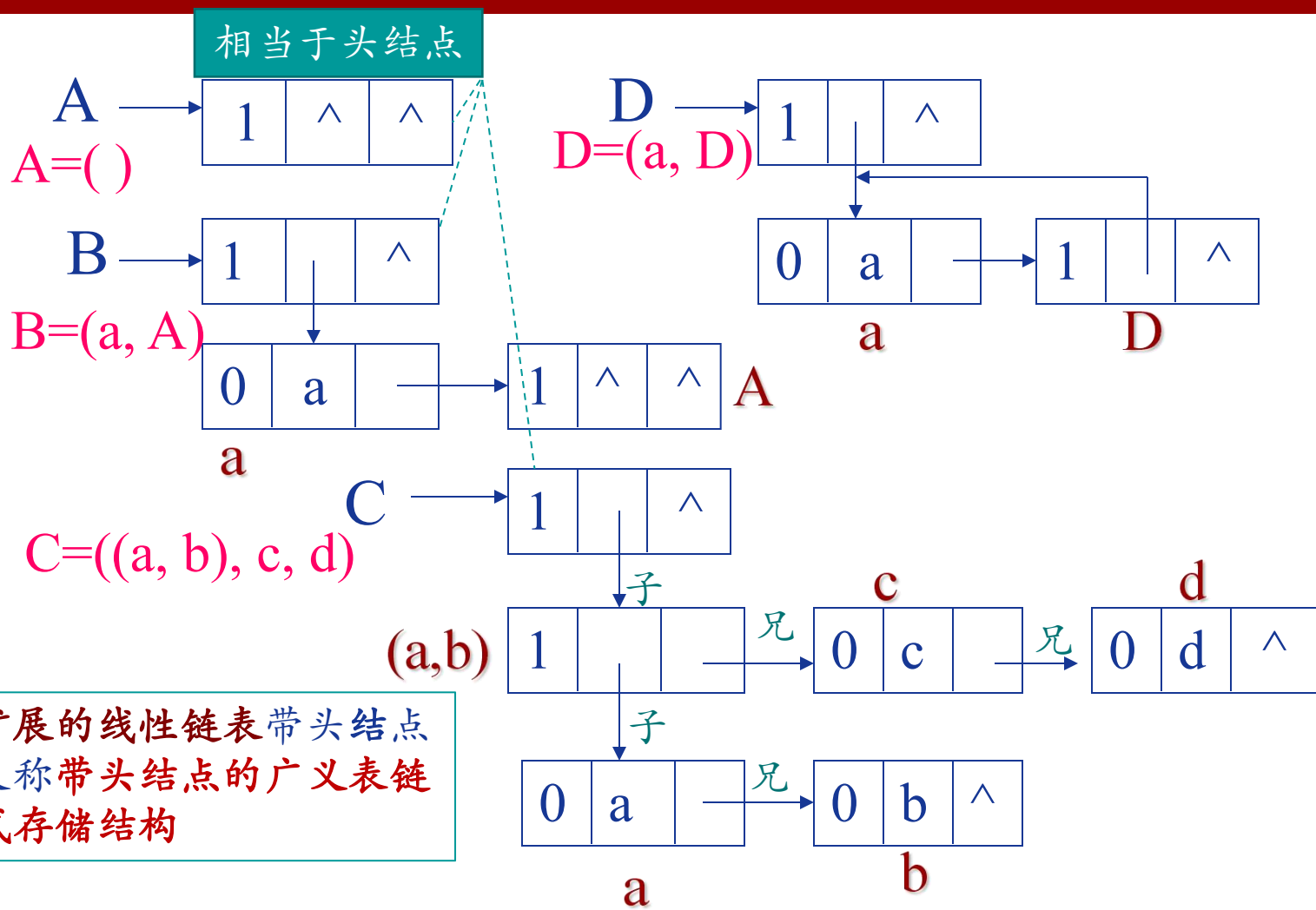
hp指向孩子; tp指向兄弟;

原子结点对应着叶子结点;

表结点对应着非叶子结点

[示例]

BUPT



5.6 广义表的递归算法

BUPT

广义表的特点：定义是递归的。

示例约定：非递归表且无共享子表。

例一 求广义表的深度

思路

- ①将广义表分解成 n 个子表，分别(递归)求得每个子表的深度
- ②广义表的深度 = $\text{Max} \{ \text{子表的深度} \} + 1$
- ③可以直接求解的两种简单情况为：

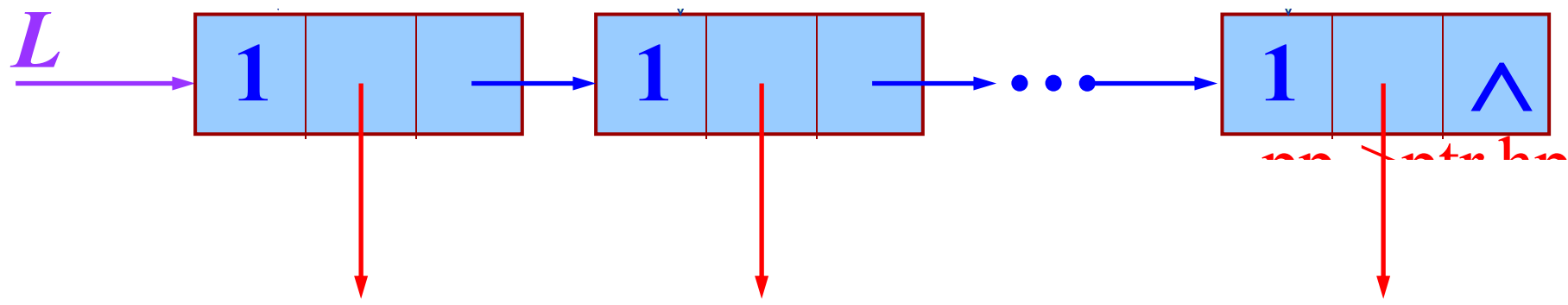
空表的深度 = 1

原子的深 = 0

【头尾链表】

```
int GlistDepth(Glist L) {  
    // 返回指针L所指的广义表的深度  
    if (!L) return 1;  
    if (L->tag == ATOM) return 0;  
  
    for (max=0, pp=L; pp!= NULL; pp=pp->ptr.tp)  
    {  
        dep = GlistDepth(pp->ptr.hp);  
        if (dep > max) max = dep;  
    }  
    return max + 1;  
} // GlistDepth
```

例如



```
for (max=0, pp=L; pp!= NULL; pp=pp->ptr.tp)
{
    dep = GlistDepth(pp->ptr.hp);
    if (dep > max) max = dep;
}
```

【扩展的线性链表】

```

int GLDepth(GLNode *g) //求带头节点的广义表g的深度
{
    int max=0,dep;
    if (g->tag==0) return 0; //为原子时返回0
    g=g->hp;    //g指向第一个元素
    if (g==NULL) return 1; //为空表时返回1
    while (g!=NULL) //遍历表中的每一个元素
    {
        if (g->tag==1)          //元素为子表的情况
        {
            dep=GLDepth(g); //递归调用求出子表的深度
            if (dep>max) max=dep;
            //max为同一层所求过的子表中深度的最大值
        }
        g=g->tp; //使g指向下一个元素
    }
    return(max+1); //返回表的深度
}

```

例二 复制广义表【头尾链表】

思路

- ①将广义表分解成表头和表尾两部分，分别(递归)复制求得新的表头和表尾，新的广义表由新的表头和表尾构成。
- ②可以直接求解的两种简单情况为：
空表复制求得的新表自然也是空表；
原子结点可以直接复制求得。

若 $ls = \text{NULL}$ 则 $\text{newls} = \text{NULL}$

否则

构造结点 newls ,

由表头 $ls \rightarrow \text{ptr.hp}$ 复制得 newhp

由表尾 $ls \rightarrow \text{ptr.tp}$ 复制得 newtp

并使 $\text{newls} \rightarrow \text{ptr.hp} = \text{newhp}$,

$\text{newls} \rightarrow \text{ptr.tp} = \text{newtp}$

```

Bool CopyGList(Glist &T, Glist L)
{
    if (!L) T = NULL; // 复制空表
    else
    {
        if ( !(T = (Glist)malloc(sizeof(GLNode))) ) // 建表结点
            return FALSE;
        T->tag = L->tag;
        if (L->tag == ATOM)
            T->data = L->data; // 复制单原子结点
        else // 分别复制表头和表尾
        {
            CopyGList(T->ptr.hp, L->ptr.hp);
            CopyGList(T->ptr.tp, L->ptr.tp);
        }
    }
    return TRUE;
}

```

1. 设计算法，给定两个均含有 n 个元素的整形数组 A 和 B ，判定 B 是否是 A 循环右移若干位得到的。

`bool isRightMove(int A[], int B[], int n)`

2. 设对称矩阵 A 如下所示：(1) 若将 A 视为对称矩阵，画出对其压缩存储的存储表，并讨论如何存取 A 中元素 a_{ij} ($1 \leq i, j \leq 4$)；
(2) 若将 A 视为稀疏矩阵，画出 A 的十字链表结构。

$$\begin{bmatrix} 2 & 0 & 0 & 4 \\ 0 & 0 & 3 & 0 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 0 & 0 \end{bmatrix}$$

3.求下列广义表操作的结果：

(1)Head[Tail[Head[((a,b),(c,d))]]]

(2)Tail[Head[Tail [((a,b),(c,d))]]]

注：[]是函数的符号

4. 利用广义表的Head和Tail操作把原子c从广义表

$L=((a,b),(c,d))$ 中分离出来，并求表L的长度和深度。

5. 假设一个稀疏矩阵t的行列数相同，采用三元组TSMatrix表示， TSMatrix结构参见课件第17页，设计算法求左上->右下对角线上非0元素之和。

bool Sum(TSMAtrix t, elemtype &sum)

\\函数返回false表示行列不等不能求和；函数返回true表示能求和，并用参数sum返回所求的和。

实验3

假设稀疏矩阵A和B均以三元组表作为存储结构，试写出矩阵相加和相乘的算法，另设三元组表C存放结果矩阵。

要求：

- 从键盘输入稀疏矩阵A和B
- 检测A和B能否相加/相乘
- 如能，做矩阵相加和相乘运算，并打印运算结果
- 如不能，应显示出原因